Learning Inverse Kinematics with Neural Networks in Webots

In this lab you will train a neural network to solve the inverse kinematics problem for a robot arm.

The model will learn to convert:

Input: Cartesian target position *(x, y, z)*

Output: Joint angles *(q1, q2, q3)*

for a 3-DOF robot arm simulated in Webots.

Instead of using traditional kinematic equations, the system will learn the mapping directly from data generated by the robot.

Provided Scripts

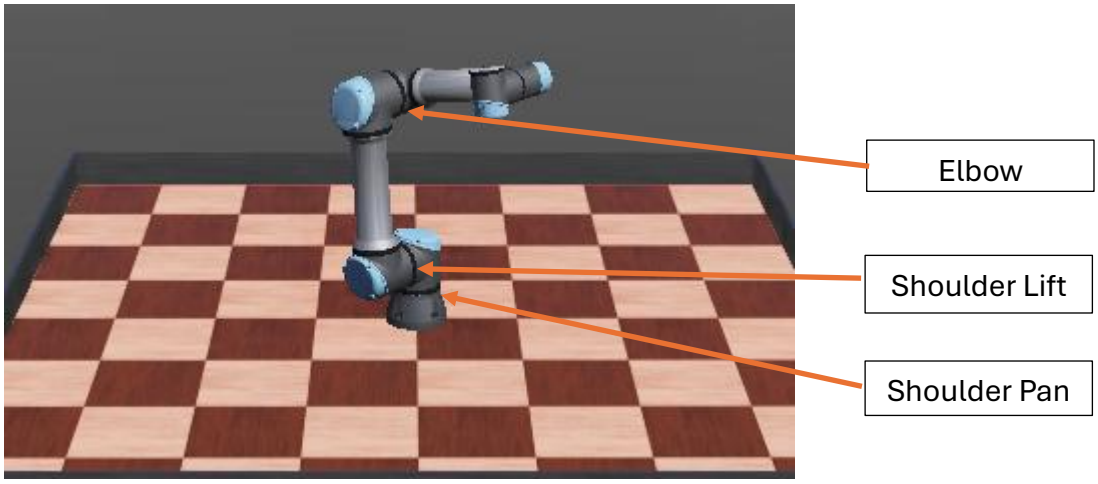
Script	Purpose
Data_generator.py	Generates training data from the robot
Trainer.py	Trains the neural network
Runner.py	Runs the trained model in Webots

System Overview

In this lab we use a modified version of the UR5 robot arm.

The original wrist modules have been removed, resulting in a 3-DOF manipulator with the following joints:

1. Shoulder Pan
2. Shoulder Lift
3. Elbow



The neural network learns the following mapping:

Input (x, y, z) Cartesian coordinates of the target position.

Output: (q1, q2, q3) joint angles required for the robot to reach the target.

1. Installation and Setup

Step 1 – Install Webots

Download Webots R2023b from:

https://github.com/cyberbotics/webots/releases/tag/R2023b

Install the version for your operating system and follow the installation instructions.

OS	File(s)
Windows 10 Intel/AMD 64-bit	webots-R2023b_setup.exe
Linux Intel/AMD 64-bit	webots_2023b_amd64.deb (Ubuntu 20.04 & 22.04) webots-R2023b-x86-64.tar.bz2 (Ubuntu 20.04 & 22.04) Webots latest/stable R2025a (Linux snap package) docker hub (Docker Image)
macOS Apple Silicon & Intel	webots-R2023b.dmg

Step 2 – Clone the Repo

Clone the git repository URL or download form OneDrive, to your local machine.

Your directory structure should look similar to:

```
controllers/
├─ ik_controller/
│  ├─ ik_controller.py
├─ nnik_trainer/
│  ├─ IK_Trainer.py
├─ random_dataset_generator/
│  ├─ random_dataset_generator.py
libraries/
pliguns/
protos/
worlds/
├─ 3DOF_Arm.wbt
NNIK_lab_sheet.pdf
```

Step 3 - Open the Webots World

1. Launch Webots
2. Select File → Open World
3. Navigate to the project folder
4. Open the worlds/3DOF_arm.wbt world file

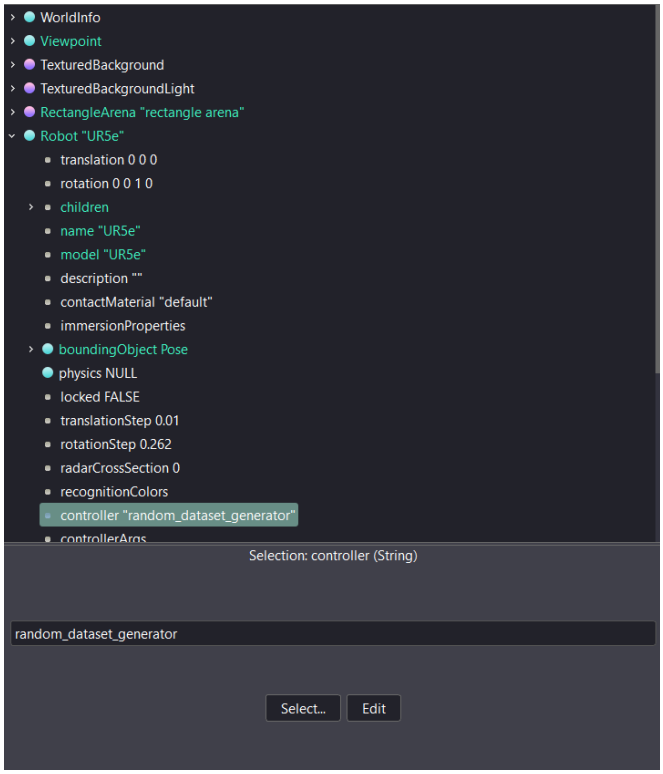
2. Dataset Generation

Step 1 — Select the Data Generator

1. In the Webots Scene Tree, locate the UR5 robot.
2. In the controller field, ensure data_generator.py is selected.
3. Press Edit to open the script.

Step 2 — Modify Dataset Size

Find the parameter TARGET_SAMPLES



Step 3 — Generate Dataset A

Find the parameter “TARGET_SAMPLES”

Generate Dataset A

Setting “TARGET_SAMPLES = 5000”

```
# ================= CONFIG ================

TARGET_SAMPLES = 5000
SETTLE_STEPS = 30
OUTPUT_FILE = "4DOF_ik_dataset_gps.csv"

```

Run in Webots to generate the dataset

Step 3 — Generate Dataset B

Set “TARGET_SAMPLES = 20000”

Run in Webots to generate the dataset like before.

Dataset Format

The resulting dataset will be saved as a **CSV file** containing rows in the following format:

(x, y, z, q1, q2, q3, q4)

Example

	A	B	C	D	E	F
1	x	y	z	q1	q2	q3
2	-0.38481	-0.09261	0.362825	2.4421583	-4.17168	-0.7956
3	-0.03188	-0.08437	0.5782980	4.2696502	-1.35881	-1.46553

3. Train the Neural Network

Open trainer.py. The default model uses the architecture shown above. Train the model on both Dataset A and Dataset B, noting training loss and time. The architecture currently has this architecture:

```
self.model = nn.Sequential(
    nn.Linear(3, 64),
    nn.ReLU(),

    nn.Linear(64, 64),
    nn.ReLU(),

    nn.Linear(64, 64),
    nn.ReLU(),

    nn.Linear(64, 4)
)
```

Train the model using each of your generated datasets

At then end of the training a Ik_model_4dof.pth will be produced.

Rename to an appropriate name for the model structure and training dataset size and move to the directory where runner.py is located

Next modify the network architecture and test one alternative network of your choice this may be different sized layer or an additional hidden layer. Note: don't make to big as a large network takes longer to train.

Make a note of the training loss and training time for all configurations

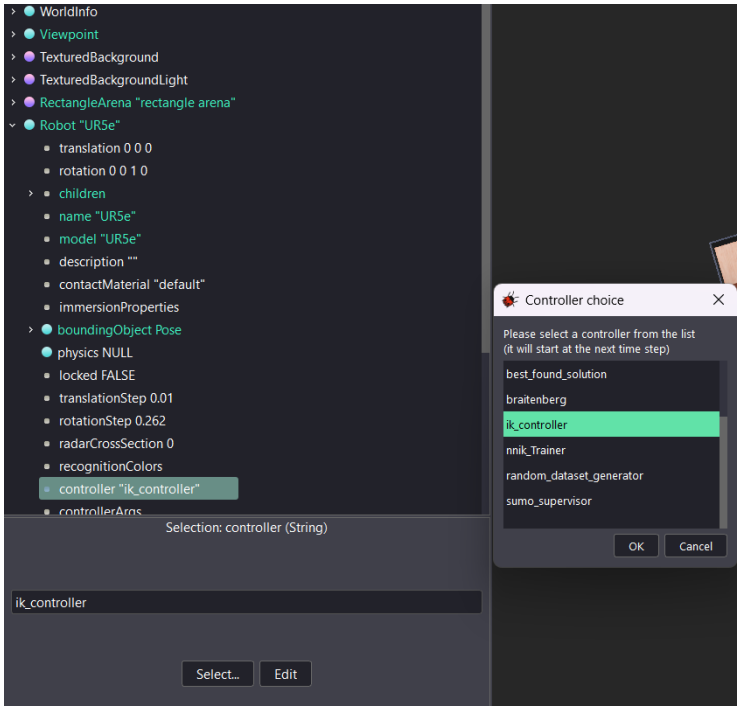
Run	Architecture	Dataset	Loss	Training Time
1	3->64->64->4	A (5k)		
2	3->64->64->4	B (20k)		
3	Custom	A (5k)		

4	Custom	B (20k)		

4. Test the models in Webots

Switch the robot controller from data_generator.py to runner.py. Update the model filename inside the script to match your saved .pth file.

1. Expand the robot UR5e in the scene tree
2. Find the variable Controller
3. Press select and find ik_controller in the menu



4. Press edit after selecting and change “MODEL_PATH” to the name of your model

```
MODEL_PATH = "ik_model_3dof.pth" # Change the model name here
```

This controller will:

1. Loads the trained model
2. Reads the position of the target ball
3. Predicts joint angles
4. Moves the robot arm

Change the name of the model file to the name of your model file and run the simulation

Move the red ball around and observe how the robot arm moves

5. Challenge

Try to improve the robot arm by modifying one on the following

1. Increase the Dataset size. Do you see a difference in the model
2. Changing the network size or shape and see what changes about the final model
3. Change the loss function to a MSELoss or a L1Loss and see what produces a better result